

Instrucciones detalladas para Lucas - Proyecto AreplA

Guia paso a paso para crear una contribucion individual, subirla a GitHub y dejar evidencia academica en commits, rama y Pull Request.

Dato	Valor
Repositorio	https://github.com/Raiden1010RV/AreplA.git
Responsable	Lucas
Tarea asignada	Arquitectura y diagramacion del proyecto
Rama de trabajo	feature-lucas-arquitectura
Carpeta principal	docs/diagramas/
Resultado esperado	Commit + Push + Pull Request visible en GitHub

Importante: todos los comandos se deben ejecutar desde CMD, PowerShell o la terminal integrada de VS Code. Si un comando falla, no continuar al siguiente paso; revisar la seccion de errores frecuentes al final.

1. Preparacion inicial

Antes de trabajar, Lucas debe confirmar tres cosas: que tiene Git instalado, que tiene acceso al repositorio de GitHub y que sabe en que carpeta guardara el proyecto en su computador.

1.1 Verificar Git instalado

```
git --version
```

Resultado esperado: debe aparecer una version de Git, por ejemplo git version 2.xx.x. Si aparece que Git no se reconoce, debe instalar Git y reiniciar la terminal.

1.2 Elegir carpeta de trabajo

Lucas puede guardar el proyecto en Documentos, Escritorio o una carpeta academica. Debe ubicarse en la carpeta superior donde quiere clonar el repositorio.

```
cd "%USERPROFILE%\Documents"
```

Si usa una ruta con espacios, debe escribirla entre comillas. Ejemplo:

```
cd "C:\Users\Raiden AsusROG\OneDrive\Documentos\Archivo Academico\Politecnico Grancolombiano - Ing. de Sistemas\8 Semestre\Integracion continua"
```

No debe crear manualmente la carpeta AreplA si va a usar git clone, porque Git la crea automaticamente.

2. Clonar el repositorio

Clonar significa descargar una copia completa del repositorio de GitHub al computador de Lucas. Desde la carpeta donde quiera guardar el proyecto, ejecutar:

```
git clone https://github.com/Raiden1010RV/AreplA.git
```

Luego entrar a la carpeta del repositorio:

```
cd AreplA
```

Verificar que si esta dentro del repositorio:

```
git status
```

Resultado esperado:

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Si aparece fatal: not a git repository, significa que Lucas no esta dentro de la carpeta AreplA. Debe ejecutar cd AreplA y repetir git status.

3. Actualizar la rama principal

Antes de crear su rama, Lucas debe asegurarse de tener lo ultimo que hicieron sus companeros.

```
git checkout main
git pull origin main
```

Resultado esperado: Git debe indicar que se actualizaron archivos o que todo esta al dia. Si pide resolver conflictos, detenerse y pedir apoyo antes de continuar.

4. Crear la rama de Lucas

Lucas no debe trabajar directamente en main. Debe crear una rama propia para que su aporte quede separado y documentado.

```
git checkout -b feature-lucas-arquitectura
```

Verificar en que rama esta:

```
git branch
```

Resultado esperado: debe verse un asterisco antes de la rama de Lucas:

```
* feature-lucas-arquitectura
main
```

5. Crear la estructura asignada

La responsabilidad de Lucas es aportar documentacion de arquitectura y diagramacion. Debe crear estos archivos:

```
docs\diagramas\arquitectura.md
docs\diagramas\flujo-datos.md
docs\diagramas\componentes.md
docs\equipo\lucas.md
```

Comandos recomendados en Windows:

```
mkdir docs\diagramas
mkdir docs\equipo
```

Si alguna carpeta ya existe, Windows puede mostrar un mensaje indicando que ya existe. Eso no es grave; puede continuar.

6. Crear el archivo de evidencia personal

Este archivo sirve para identificar la contribucion individual de Lucas.

```
echo # Lucas > docs\equipo\lucas.md
echo Responsable de arquitectura y diagramacion del proyecto AreplA. >> docs\equipo\lucas.md
echo Rama de trabajo: feature-lucas-arquitectura >> docs\equipo\lucas.md
```

7. Crear documento de arquitectura

Ejecutar los siguientes comandos para crear contenido minimo pero util en arquitectura.md:

```

echo # Arquitectura ArepIA > docs\diagramas\arquitectura.md
echo Responsable: Lucas >> docs\diagramas\arquitectura.md
echo. >> docs\diagramas\arquitectura.md
echo Objetivo: documentar la estructura general del sistema ArepIA. >> docs\diagramas\arquitectura.md
echo. >> docs\diagramas\arquitectura.md
echo Componentes principales: >> docs\diagramas\arquitectura.md
echo - Frontend: interfaz de usuario. >> docs\diagramas\arquitectura.md
echo - Backend: API y logica de negocio. >> docs\diagramas\arquitectura.md
echo - Base de datos: almacenamiento de ventas, productos e inventario. >> docs\diagramas\arquitectura.md
echo - Modelo IA: prediccion de demanda y stock recomendado. >> docs\diagramas\arquitectura.md
echo - GitHub Actions: integracion continua. >> docs\diagramas\arquitectura.md

```

8. Crear documento de flujo de datos

Este archivo explica de forma simple como viaja la informacion en el sistema.

```

echo # Flujo de datos ArepIA > docs\diagramas\flujo-datos.md
echo Responsable: Lucas >> docs\diagramas\flujo-datos.md
echo. >> docs\diagramas\flujo-datos.md
echo 1. El usuario registra ventas e inventario desde el frontend. >> docs\diagramas\flujo-datos.md
echo 2. El backend recibe la informacion mediante servicios API. >> docs\diagramas\flujo-datos.md
echo 3. La base de datos almacena ventas, productos e insumos. >> docs\diagramas\flujo-datos.md
echo 4. El modulo IA analiza datos historicos y variables externas. >> docs\diagramas\flujo-datos.md
echo 5. El sistema entrega una recomendacion de produccion para el dia siguiente. >> docs\diagramas\flujo-datos.md

```

9. Crear documento de componentes

Este archivo describe que hace cada parte del proyecto.

```

echo # Componentes ArepIA > docs\diagramas\componentes.md
echo Responsable: Lucas >> docs\diagramas\componentes.md
echo. >> docs\diagramas\componentes.md
echo Frontend: permite consultar inventario, registrar ventas y ver recomendaciones. >> docs\diagramas\componentes.md
echo Backend: gestiona reglas de negocio, servicios y conexion con base de datos. >> docs\diagramas\componentes.md
echo Base de datos: conserva informacion del negocio. >> docs\diagramas\componentes.md
echo Modelo IA: calcula demanda estimada a partir de datos historicos. >> docs\diagramas\componentes.md
echo Integracion continua: valida cambios mediante GitHub Actions. >> docs\diagramas\componentes.md

```

10. Revisar cambios antes de confirmar

Lucas debe revisar que archivos cambio antes de hacer commit.

```
git status
```

Resultado esperado: deben aparecer archivos nuevos en rojo o verde, por ejemplo:

```

new file: docs/diagramas/arquitectura.md
new file: docs/diagramas/flujo-datos.md
new file: docs/diagramas/componentes.md
new file: docs/equipo/lucas.md

```

Opcional: abrir el proyecto en VS Code para revisar los archivos:

```
code .
```

11. Preparar archivos para commit

Agregar todos los archivos creados.

```
git add .
```

Verificar que quedaron preparados:

```
git status
```

Resultado esperado: deben aparecer como Changes to be committed.

12. Crear commit individual

El commit debe tener un mensaje claro porque sera usado como evidencia academica.

```
git commit -m "Agrega documentacion de arquitectura ArepIA"
```

Resultado esperado: Git debe mostrar algo parecido a:

```
[feature-lucas-arquitectura abc1234] Agrega documentacion de arquitectura ArepIA
4 files changed, XX insertions(+)
```

13. Subir la rama a GitHub

Como la rama fue creada localmente, hay que subirla a GitHub.

```
git push origin feature-lucas-arquitectura
```

Resultado esperado: Git debe indicar que la rama fue subida. En GitHub debe aparecer un boton que dice Compare & pull request.

14. Crear Pull Request

En GitHub, Lucas debe seguir esta ruta:

Repositorio ArepIA -> Pull requests -> New pull request

Configurar asi:

Campo	Valor
base	main
compare	feature-lucas-arquitectura
Titulo	Agrega documentacion de arquitectura ArepIA
Descripcion	Se agregan documentos iniciales de arquitectura, flujo de datos, componentes y evidencia personal de Lucas.

Despues debe hacer clic en Create pull request. No debe hacer merge sin avisar si el equipo quiere revisar primero.

15. Evidencia que debe entregar Lucas

Evidencia	Donde encontrarla
Rama creada	GitHub -> Branches o Pull Request
Commit individual	GitHub -> Code -> Commits
Pull Request	GitHub -> Pull requests
Archivos creados	GitHub -> docs/diagramas y docs/equipo
Contribucion personal	GitHub -> Insights -> Contributors

16. Errores frecuentes y solucion

Error	Causa probable	Solucion
fatal: not a git repository	No esta dentro de AreplA	Ejecutar cd AreplA y luego git status
docs no se reconoce como comando	Intento ejecutar una ruta como comando	Usar code docs\archivo.md para abrir o echo para escribir
Everything up-to-date despues de modificar	Falto hacer commit	Ejecutar git add, git commit y luego git push
Permission denied o authentication failed	No tiene acceso al repo o no inicio sesion	Aceptar invitacion a GitHub y verificar credenciales
pathspec main did not match	No existe rama main local o no esta clonado bien	Revisar git branch y pedir apoyo
Merge conflict	Dos personas modificaron el mismo archivo	Detenerse y pedir apoyo antes de borrar contenido

17. Checklist final para Lucas

☐ Estoy dentro de la carpeta AreplA.

☐ Ejecute git pull origin main antes de trabajar.

☐ Cree la rama feature-lucas-arquitectura.

☐ Cree los archivos en docs/diagramas y docs/equipo.

☐ Ejecute git status y revise cambios.

☐ Ejecute git add .

☐ Ejecute git commit con mensaje claro.

☐ Ejecute git push origin feature-lucas-arquitectura.

☐ Cree el Pull Request hacia main.

☐ Envie captura del commit, PR y archivos creados.

18. Comandos resumidos para copiar y pegar

Usar este bloque solo si ya entendio los pasos anteriores:

```
git clone https://github.com/Raiden1010RV/AreplA.git
cd AreplA
git checkout main
git pull origin main
git checkout -b feature-lucas-arquitectura
mkdir docs\diagramas
mkdir docs\equipo
echo # Lucas > docs\equipo\lucas.md
echo Responsable de arquitectura y diagramacion del proyecto AreplA. >> docs\equipo\lucas.md
echo # Arquitectura AreplA > docs\diagramas\arquitectura.md
echo Responsable: Lucas >> docs\diagramas\arquitectura.md
echo # Flujo de datos AreplA > docs\diagramas\flujo-datos.md
echo Responsable: Lucas >> docs\diagramas\flujo-datos.md
echo # Componentes AreplA > docs\diagramas\componentes.md
echo Responsable: Lucas >> docs\diagramas\componentes.md
git status
git add .
git commit -m "Agrega documentacion de arquitectura AreplA"
git push origin feature-lucas-arquitectura
```

Fin de la guía. Si Lucas llega hasta el Pull Request, su participación quedará documentada en commits, rama, archivos modificados y evidencia visual dentro de GitHub.